MailScope

Email Deliverability Analysis Using DNS and MX Records

Internship Project Submission

Live Application: https://mail-scope-zeta.vercel.app

GitHub Repository: https://github.com/Dhanurdhar21/mailScope

Backend API: https://mailscope-backend.onrender.com

API Documentation: https://mailscope-backend.onrender.com/docs

Health Check: https://mailscope-backend.onrender.com/health

React • TypeScript • FastAPI • dnspython

1. Project Overview

MailScope is a web application that assesses email-domain deliverability using email syntax validation, DNS resolution, and Mail Exchange (MX) record analysis. It combines a React and TypeScript frontend with a Python FastAPI backend. It presents classifications and technical explanations to help users understand whether a domain appears configured to receive email.

MailScope performs analysis without sending test emails. DNS and MX records provide useful evidence about a domain's mail-routing configuration, but they cannot prove that a specific mailbox exists or guarantee that a message will reach the recipient.

2. Problem Statement and Objectives

An email address can look correctly formatted while using a nonexistent domain or a domain that is not configured to receive email. Manually checking many addresses and their DNS configuration can be inconvenient.

- Validate email address formatting before domain-level analysis.
- Inspect DNS responses and MX records to understand mail-routing configuration.
- Distinguish likely configuration problems from temporary or inconclusive DNS failures.
- Present results with readable classifications and technical explanations.
- Support batch analysis and CSV export.

3. Main Features

Email syntax validation

The backend checks address formatting and identifies common problems such as a missing '@' delimiter, invalid dot placement, and malformed domain components. Syntax validity does not establish that a mailbox exists.

DNS and domain analysis

MailScope uses dnspython to query DNS information. It interprets responses such as successful resolution, NXDOMAIN, and DNS lookup failures. A timeout is not proof that a domain does not exist.

MX record and fallback analysis

MX records identify mail servers designated to receive email for a domain. MailScope distinguishes ordinary MX records, Null MX (which explicitly indicates that the domain does not accept email), and missing-MX scenarios. Where applicable, A/AAAA records may be considered as potential implicit-routing fallback information.

Classification and technical explanation

The application uses three primary classifications: likely_to_be_delivered, likely_to_bounce, and unknown. Its backend maps available syntax and DNS evidence to these application-level labels and returns technical reasoning where available.

Batch analysis, CSV export, and health check

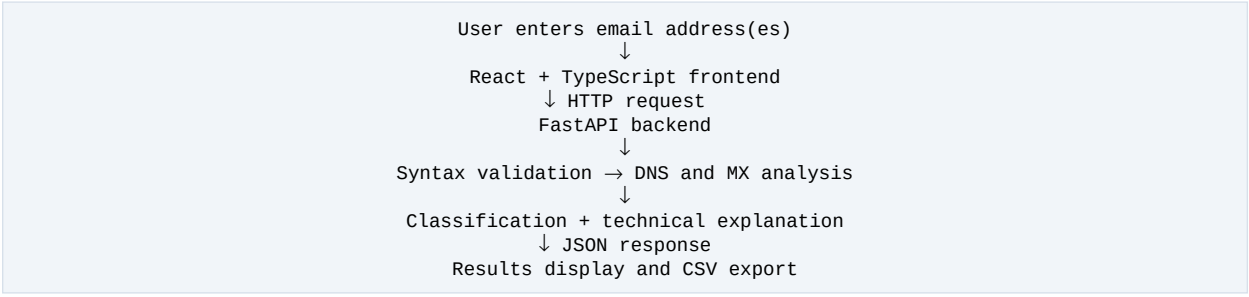
The application supports batch analysis with a configured maximum of 200 addresses, CSV export of results, and a backend health endpoint at GET /health. A successful health check does not guarantee every analysis request will succeed.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React, TypeScript, Vite	User interface and frontend build
Styling/UI	Tailwind CSS, CSS, Lucide React	Layout, styling, icons
Backend	Python, FastAPI, Uvicorn	HTTP API and request handling
DNS	dnspython	DNS and MX queries
Validation	Pydantic, idna	Request/response validation and domain-name handling
Testing	pytest and related tools	Backend tests

5. System Architecture

MailScope follows a frontend-backend architecture. The browser sends analysis requests to the FastAPI backend. The backend validates input, extracts the domain, performs DNS/MX analysis, interprets the available evidence, and returns structured JSON results for the frontend to display.



6. How the Analysis Works

1. Input: the user enters one or more email addresses.
2. Syntax validation: the backend checks each address against its validation rules. Invalid syntax can be rejected before DNS checks.
3. Domain extraction: the domain portion after '@' is identified.
4. DNS resolution: the backend interprets DNS responses, distinguishing domain nonexistence from timeouts and other resolution errors.
5. Mail-routing analysis: MX records are inspected. Null MX and missing-MX scenarios are treated differently; applicable A/AAAA fallback information may also be considered.
6. Classification: the backend applies its rules to the evidence and selects likely_to_be_delivered, likely_to_bounce, or unknown.
7. Response: structured results are returned to the frontend for display and possible CSV export.

7. Classification Guide

Classification	Interpretation
likely_to_be_delivered	Evidence supports the possibility of delivery, such as valid syntax and domain mail-routing configuration. It does not verify the recipient mailbox.
likely_to_bounce	Evidence suggests a delivery problem, such as malformed syntax, a confirmed nonexistent domain, or explicit Null MX.
unknown	Evidence is insufficient or ambiguous, for example due to DNS timeouts or temporary resolver failures.

These labels are assessments, not guarantees. Actual delivery depends on mailbox existence, recipient-server policies, sender reputation, authentication, and message filtering.

8. How to Run Locally

Prerequisites: install a compatible Python version, Node.js, and npm. Use dependency versions specified by the repository configuration.

Step 1 — Install backend dependencies

```
python -m pip install -r backend/requirements.txt
```

Step 2 — Configure the frontend API URL

Create or update the Vite environment file used by the project (commonly .env.local):

```
VITE_API_BASE_URL=http://localhost:8000
```

Configure backend variables according to backend/config.py and .env.example. Do not commit secrets.

Step 3 — Start the backend

```
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
```

The local backend should be available at http://127.0.0.1:8000, with interactive API documentation typically at http://127.0.0.1:8000/docs.

Step 4 — Install and start the frontend

```
npm install
npm run dev
```

Open the local URL shown by Vite, typically http://localhost:5173. Restart the development server after changing Vite environment variables.

9. API Endpoints

Method	Endpoint	Purpose
GET	/health	Checks whether the backend health endpoint responds.
POST	/api/analyze	Submits email addresses for syntax and DNS/MX analysis.

Use the live Swagger/OpenAPI documentation for the current request and response schemas. Example output can vary with the input and live DNS responses.

10. Testing

The repository README describes backend tests for syntax rules, API behavior, and DNS scenarios. From the repository root, run:

```
python -m pytest -v
```

If live DNS integration tests are marked separately, deterministic tests can be selected with:

```
python -m pytest -m "not integration" -v
```

Report test counts and pass/fail claims only after running tests in the current environment. Live DNS tests can depend on network connectivity and external DNS infrastructure.

11. Deployment and Links

Resource	Link
Live frontend	https://mail-scope-zeta.vercel.app
GitHub repository	https://github.com/Dhanurdhar21/mailScope
Backend service	https://mailscope-backend.onrender.com
Interactive API documentation	https://mailscope-backend.onrender.com/docs
Backend health endpoint	https://mailscope-backend.onrender.com/health

The frontend is deployed on Vercel and the backend on Render. The production frontend should use VITE_API_BASE_URL pointing to the deployed backend, and the backend CORS configuration should allow the deployed frontend origin.

12. Limitations

- DNS/MX analysis cannot reliably establish whether an individual recipient mailbox exists.
- A domain with MX records may still reject a recipient or message.
- A positive classification does not guarantee acceptance, inbox placement, or successful delivery.
- A negative classification is an assessment, not proof that every message will bounce.
- DNS timeouts and temporary resolver failures can make a result inconclusive.
- Spam filtering, sender reputation, authentication policies, and recipient-server rules are not determined with certainty by DNS-only analysis.

13. Conclusion

MailScope combines a web interface, backend API, syntax validation, and DNS/MX analysis to provide an understandable assessment of email-domain configuration. Its value is making available evidence visible and explaining the reasoning behind classifications while communicating the limitations of DNS-based verification.